

Software Requirements Specification (SRS)

Stock Portfolio Tracker Web Application

Version: 1.0

Date: July 4, 2025

Prepared by: [David Jesse Odhiambo](#)

1. Introduction

1.1 Purpose

This SRS defines the functional, non-functional, and user interface requirements for a Stock Portfolio Tracker web application. The application allows users to search for stock tickers, view real-time price data, simulate trades, and track portfolio performance. The document serves as a contract between stakeholders (clients, end-users, developers) to ensure the system meets user expectations and adheres to real-world constraints.

1.2 Scope

The Stock Portfolio Tracker is a web-based application that enables users to:

- Search for stock tickers (e.g., AAPL, TSLA) using an external financial API (e.g., Finnhub.io, Twelve Data, or Alpha Vantage).
- View stock details including company name, current stock price, and percentage change.
- Simulate buying and selling shares, managed in local state.
- Display the total portfolio value and gain/loss metrics.
- Provide a responsive, user-friendly interface for seamless interaction.

The application will be developed with a focus on usability, performance, and maintainability, hosted on a web platform accessible via modern browsers. It will integrate with third-party APIs for real-time stock data and simulate trades without actual financial transactions.

1.3 Definitions, Acronyms, and Abbreviations

- **SRS:** Software Requirements Specification
- **API:** Application Programming Interface
- **UI:** User Interface
- **KLOC:** Thousand Lines of Code
- **SDLC:** Software Development Life Cycle
- **Stock Ticker:** A unique symbol representing a publicly traded company (e.g., AAPL for Apple Inc.)
- **Portfolio:** A collection of simulated stock holdings for a user
- **Gain/Loss:** The difference between the current value of the portfolio and the initial investment

2. Overall Description

2.1 User Needs

The primary users are individual investors or finance enthusiasts who want to:

- Explore stock market data in real-time.
- Simulate investment strategies without financial risk.
- Track portfolio performance metrics (total value, gain/loss).
- Use an intuitive interface to manage their simulated investments.

2.2 Assumptions and Constraints

Assumptions:

- Users have access to modern web browsers (e.g., Chrome, Firefox, Safari).
- Third-party APIs (Finnhub.io, Twelve Data, or Alpha Vantage) are reliable and available.
- Users are familiar with basic stock market concepts (e.g., tickers, prices, percentage change).

Constraints:

- **Financial:** Limited budget for API subscriptions; preference for free-tier APIs with usage quotas.
- **Technical:** The application relies on third-party APIs, which may have rate limits or downtime.
- **Legal:** Compliance with API terms of service and data usage policies.
- **Performance:** Must handle API response times and ensure the UI remains responsive.

2.3 System Context

The system interacts with:

- **Users:** Via a web interface to input stock tickers, view data, and simulate trades.
- **Third-Party APIs:** To fetch real-time stock data (e.g., company name, price, percentage change).
- **Local State:** To store and manage simulated portfolio data (e.g., purchased shares, quantities).

3. System Requirements

3.1 Functional Requirements

Functional requirements define the core functionalities of the Stock Portfolio Tracker. These are categorized as Must Have, Should Have, Could Have, and Wish List per the CN7021 document.

1. Must Have:

- **FR1:** The system shall allow users to input a valid stock ticker symbol (e.g., AAPL, TSLA) to search for stock information.
- **FR1:** The user should be able to authenticate successfully.
- **FR2:** The system shall display the company name, current stock price, and percentage change for a searched sticker using data from a third-party API.
- **FR3:** The system shall allow users to simulate buying shares by specifying a ticker and quantity, storing the transaction in local state.
- **FR4:** The system shall display the user's portfolio, including a list of owned stocks, quantities, purchase prices, current prices, and total portfolio value.

- **FR5:** The system shall calculate and display the total gain/loss for the portfolio based on current stock prices versus purchase prices.

2. Should Have:

- **FR6:** The system should allow users to simulate selling shares by specifying a ticker and quantity, updating the portfolio in local state.
- **FR7:** The system should provide a historical price chart for a searched stock ticker (e.g., daily, weekly, or monthly data).
- **FR8:** The system should allow users to save their portfolio locally (e.g., in browser local storage) for persistence across sessions.

3. Could Have:

- **FR9:** The system could provide an option to filter or sort the portfolio by metrics (e.g., gain/loss, stock name, or value).
- **FR10:** The system could display additional stock metrics (e.g., market cap, volume, 52-week high/low) if supported by the API.

4. Wish List:

- **FR11:** The system could support simulated short selling or options trading.
- **FR12:** The system could integrate with multiple APIs to compare data accuracy or provide fallback options.

3.2 Non-Functional Requirements

Non-functional requirements ensure the system's quality attributes and operational constraints.

1. Security:

- **NFR1:** The system shall use secure API keys (stored server-side or encrypted) to access third-party APIs.
- **NFR2:** The system shall not store sensitive user data (e.g., real financial information) as it is a simulation.

2. Performance:

- **NFR3:** The system shall retrieve and display stock data within 2 seconds of a user's search request, assuming API response time is within 1 second.
- **NFR4:** The system shall handle up to 100 + simultaneous users without performance degradation.

3. Usability:

- **NFR5:** The system shall have a simple, intuitive interface that allows users to perform core functions (search, buy, view portfolio) within 3 clicks.
- **NFR6:** The system shall provide clear error messages for invalid inputs (e.g., invalid ticker) or API failures.

4. Availability:

- **NFR7:** The system shall be available 99.9% of the time, excluding third-party API downtime.

5. Interoperability:

- **NFR8:** The system shall integrate with at least one of the suggested APIs (Finnhub.io, Twelve Data, or Alpha Vantage) via RESTful API calls.

6. Scalability:

- **NFR9:** The system shall support future integration of additional APIs or features without requiring major architectural changes.

7. Maintainability:

- **NFR10:** The system shall use modular code with clear documentation, enabling developers to update or extend functionality with minimal effort.

3.3 User Interface Requirements

The UI is critical for user acceptance and must align with the principles outlined in the CN7021 document (e.g., simplicity, responsiveness, consistency).

- **UIR1:** The system shall present stock data (company name, price, percentage change) in a clear, tabular, or card-based format.
- **UIR2:** The system shall provide an input field for users to enter stock tickers, with autocomplete suggestions (if supported by the API).
- **UIR3:** The system shall include buttons for simulating “Buy” and “Sell” actions, clearly labeled and accessible from the portfolio view.
- **UIR4:** The system shall use consistent UI elements (e.g., buttons, fonts, colors) across all pages to ensure a cohesive experience.
- **UIR5:** The system shall be responsive, supporting screen sizes from mobile (320px) to desktop (1920px).
- **UIR6:** The system shall provide help information (e.g., tooltips or a help page) explaining how to use the application.
- **UIR7:** The system shall use strategic colors (e.g., green for gain, red for loss) to enhance readability and user understanding.

3.4 System Constraints

- **Physical:** The system must be lightweight to run on standard web hosting plans (e.g., < 30GB storage).
- **Legal:** The system must comply with the terms of service of the chosen API (e.g., no redistribution of data).
- **Financial:** The system should prioritize free-tier APIs to minimize development costs, with potential upgrades to paid tiers for higher quotas.

4. Requirements Engineering Process

4.1 Feasibility Study

A feasibility study was conducted to assess the viability of the Stock Portfolio Tracker:

- **Technical Feasibility:** The system is feasible using modern web technologies (e.g., React, Node.js) and RESTful APIs. The suggested APIs (Finnhub.io, Twelve Data, Alpha Vantage) provide free tiers sufficient for prototyping.
- **Financial Feasibility:** Free-tier APIs reduce initial costs. Hosting on platforms like Vercel or Netlify is low-cost or free for small-scale applications.
- **Legal Feasibility:** Compliance with API terms ensures legal operation. No real financial transactions are involved, avoiding regulatory concerns.
- **Operational Feasibility:** The system aligns with user needs for a simple, educational tool to simulate stock trading. Recommendation: The project is feasible and should proceed to requirements elicitation.

4.2 Requirements Elicitation

The requirements were gathered using the following techniques (aligned with CN7021):

- **Interviews:** Conducted with potential users (finance enthusiasts, students) to understand desired features (e.g., real-time data, portfolio tracking).
- **Surveys:** Distributed to a small group of stakeholders to identify priorities (e.g., ease of use, performance).
- **Domain Analysis:** Analyzed similar tools (e.g., Yahoo Finance, Google Finance) to identify standard features and user expectations.
- **Prototyping:** A low-fidelity UI prototype (hand-drawn or wireframe) was created to validate the layout and core features with stakeholders.

4.3 Requirements Specification

This SRS formalizes the gathered requirements in a structured format, using:

- Natural language for user requirements (e.g., "I want to see my portfolio's total value").
- Technical language for developer requirements (e.g., API integration details, local state management).
- Pseudo-code or data flow diagrams (if needed) for complex logic (e.g., gain/loss calculations).

4.4 Requirements Validation

The requirements will be validated to ensure they are:

- **Complete:** All core features (search, trade simulation, portfolio tracking) are covered.
- **Consistent:** No conflicting requirements (e.g., UI simplicity aligns with performance goals).
- **Unambiguous:** Each requirement has a single interpretation (e.g., "current stock price" refers to the latest API-provided price).
- **Testable:** Acceptance tests will verify each functional requirement (e.g., "Given a valid ticker, the system displays the correct price").
- **Achievable:** The system can be built within the constraints of free-tier APIs and standard web technologies.

5. User Stories

User stories capture user needs in a format suitable for agile development:

1. **As a user**, I want to search for a stock ticker so that I can view its current price and percentage change.
2. **As a user**, I want to simulate buying shares so that I can build a portfolio without financial risk.
3. **As a user**, I want to see my portfolio's total value and gain/loss so that I can evaluate my investment strategy.
4. **As a user**, I want a responsive interface so that I can use the application on my phone or desktop.
5. **As a user**, I want clear error messages so that I understand when an action fails (e.g., invalid ticker).

Acceptance Tests:

- **For FR1:** Given a valid ticker (e.g., AAPL), the system retrieves and displays the company name, price, and percentage change within 2 seconds.
- **For FR3:** Given a user inputs a ticker and quantity, the system adds the transaction to the portfolio and updates the total value.
- **For FR5:** Given a portfolio with at least one stock, the system calculates and displays the correct gain/loss based on current prices.

6. Use Cases

Use Case 1: Search for Stock

Actor: User

Precondition: User is on the homepage with access to the search bar.

Scenario:

1. User enters a stock ticker (e.g., AAPL) in the search bar.
2. System validates the ticker via the API.
3. System displays the company name, current price, and percentage change.
4. Postcondition: Stock details are displayed, or an error message is shown for invalid tickers.

Exception: If the API fails or the ticker is invalid, the system displays an error message.

Use Case 2: Simulate Buy Transaction

Actor: User

Precondition: User has searched for a valid stock ticker.

Scenario:

1. User clicks the "Buy" button and enters a quantity of shares.
2. System records the transaction (ticker, quantity, current price) in local state.
3. System updates the portfolio display with the new holding.

Postcondition: Portfolio reflects the new transaction; total value is updated.

Exception: If the quantity is invalid (e.g., negative), an error message is displayed.

Use Case 3: View Portfolio

Actor: User

Precondition: User has at least one simulated transaction in the portfolio.

Scenario:

1. User navigates to the portfolio page.
2. System displays a list of stocks, quantities, purchase prices, current prices, and total gain/loss.

Postcondition: Portfolio metrics are displayed accurately.

Exception: If no transactions exist, the system displays a "Portfolio Empty" message.

7. Software Metrics

To ensure quality and control, the following metrics will be tracked:

- **Function Point Count:** Measure functionality (e.g., search, trade simulation, portfolio display).
- **Complexity Metrics:** McCabe's Cyclomatic Complexity to assess code maintainability.
- **Quality Metrics:** Number of defects found during testing and user-reported issues post-deployment.
- **Process Metrics:** Time spent on each SDLC phase (e.g., requirements, coding, testing).

8. Deliverables

The development team will provide:

GitHub Repository:

- Feature branches for each major feature (e.g., search, portfolio, trade simulation).
- Clear commit messages detailing changes.
- Pull Requests (PRs) for peer/code review.

README.md:

- Setup instructions (e.g., installing dependencies, configuring API keys).
- Screenshots of the application (e.g., search page, portfolio view).
- API usage notes (e.g., endpoints used, rate limits).

Prototype:

A functional web application deployed on a hosting platform (e.g., Vercel, Netlify).

References

- CN7021- Advanced Software Engineering Document
- Sommerville, I. (2016). Software Engineering. 10th Edition.
- Pressman, R. S. & Maxim, B. R. (2015). Software Engineering: A Practitioner's Approach. 8th Edition.
- API Documentation: Finnhub.io, Twelve Data, Alpha Vantage